RTOS TWI

开发指南

版本号: 1.12

发布日期: 2025.7.28

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.7.16	AWA1637	初始版本
1.1	2020.4.27	AWA1637	添加 F133 的说明
1.2	2022.10.28	XAA0248	添加 R128 的说明
1.3	2023.3.16	XAA0309	添加 MR527 的说明
1.4	2023.3.20	XAA0248	添加 XR875 的说明
1.5	2023.5.30	XAA0309	添加 AI985 的说明
1.6	2023.11.20	XAA0309	添加 T113_S3P/S4/S4P 的说明
1.7	2023.12.19	XAA0311	根据 MR527 评审意见修改文档
1.8	2024.8.21	AWA2215	添加 MR536 的说明
1.9	2024.8.30	AWA2215	添加 T536 的说明
1.1	2024.10.11	AWA2138	1. 添加 V821 的说明 2. 丰富模块使用范例介绍
1.11	2025.7.18	AWA1669	添加 F135 的说明
1.12	2025.7.28	AWA2398	添加 MR153 的说明

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

目录

第 1 章 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 标志说明 1
1.4.2 地址与数据描述方法约定 1
1.4.3 数值单位约定 2
1.5 相关术语介绍 2
1.5.1 硬件术语 2
1.5.2 软件术语 2

第 2 章 模块介绍 3
2.1 模块功能 3
2.2 模块配置 3
2.2.1 sys_config.fex 配置说明 3
2.2.2 menuconfig 配置说明 4
2.3 源码结构 4
2.4 软件框架 5

第 3 章 模块接口说明 6
3.1 数据结构 6
3.1.1 TWI 端口号 twi_port_t 6
3.1.2 TWI 消息结构体 twi_msg_t 6
3.1.3 TWI 控制结构体 hal_twi_transfer_cmd_t 7
3.2 TWI 初始化接口 7
3.3 TWI 初始化接口（带 freq） 7
3.4 TWI 反初始化接口 7
3.5 TWI 配置从机模式接口 7
3.6 TWI 配置从机模式回调函数 8
3.7 TWI 反配置从机模式接口 8
3.8 TWI 从机模式重置接口 8
3.9 TWI 数据发送接口 v1 8
3.10 TWI 数据发送接口 v2 9
3.11 TWI 数据接收接口 9
3.12 TWI 数据控制接口 9
3.13 twi 传输接口 9
3.14 TWI 获取工作模式接口 10
3.15 TWI 设置工作模式接口 10
3.16 TWI 获取 scl 时钟接口 10
3.17 TWI 设置 scl 时钟接口 10

第 4 章 功能开发 11
4.1 TWI 做为主机读写流程 11
4.1.1 开发流程 11
4.1.2 编程示例 11
4.2 TWI 做为从机读写流程 11
4.2.1 开发流程 11
4.2.2 编程示例 12

第 5 章 日志分析 13
5.1 调试信息 13
5.2 调试工具 13

第 6 章 FAQ 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

6.1 常见问题排查 . . . 15
6.1.1 无法发送 start 信号 . . . 15
6.1.1.1 问题表现 . . . 15
6.1.1.2 可能原因和排查方法 . . . 15
6.1.2 无 ack 应答 . . . 16
6.1.2.1 问题表现 . . . 16
6.1.2.2 可能原因和排查方法 . . . 16
6.1.3 无法发送 stop 信号 . . . 16
6.1.3.1 问题表现 . . . 16
6.1.3.2 可能原因和排查方法 . . . 16
6.1.4 总线处于非空闲状态 . . . 16
6.1.4.1 问题表现 . . . 16
6.1.4.2 可能原因和排查方法 . . . 16
6.2 一号通 FAQ 列表 . . . 16

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

表格

表 1-1 适用产品列表 1
表 1-2 地址与数据描述方法约定 2
表 1-3 数值单位约定 2
表 1-4 硬件术语 2
表 1-5 软件术语 2
表 3-1 模块接口列表 6

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 1 章 前言

1.1 文档简介

介绍 RTOS 中 TWI 驱动的接口及使用方法，为 TWI 使用者提供参考。

1.2 目标读者

- TWI 驱动的开发/维护工程师。
- TWI 驱动的测试工程师。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V459	Melis	hal_twi.c
R328	FreeRTOS	hal_twi.c
R128	FreeRTOS	hal_twi.c
XR875	FreeRTOS	hal_twi.c
MR527	FreeRTOS	hal_twi.c
AI985	FreeRTOS	hal_twi.c
T113_S3P/S4/S4P	FreeRTOS	hal_twi.c
MR536	FreeRTOS	hal_twi.c
T536	FreeRTOS	hal_twi.c
V821	FreeRTOS	hal_twi.c
F135	Melis	hal_twi.c
MR153	FreeRTOS	hal_twi.c

1.4 文档约定

1.4.1 标志说明

注意

- 提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。
		例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
Sunxi	指 Allwinner 的一系列 SoC 硬件平台。
TWI	Normal Two Wire Interface，兼容 I2C 协议。

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
HAL	Hardware Abstraction Layer，硬件抽象层
RTOS	Real Time Operatiing System，实时操作系统
GPIO	General Purpose Input/Output，通用输入输出

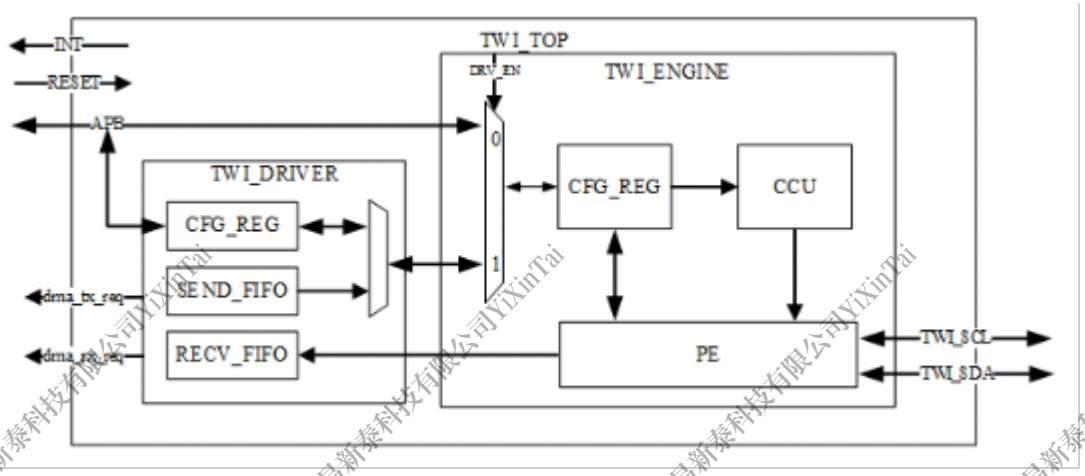
深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 2 章 模块介绍

2.1 模块功能

图 2-1: TWI 硬件框图

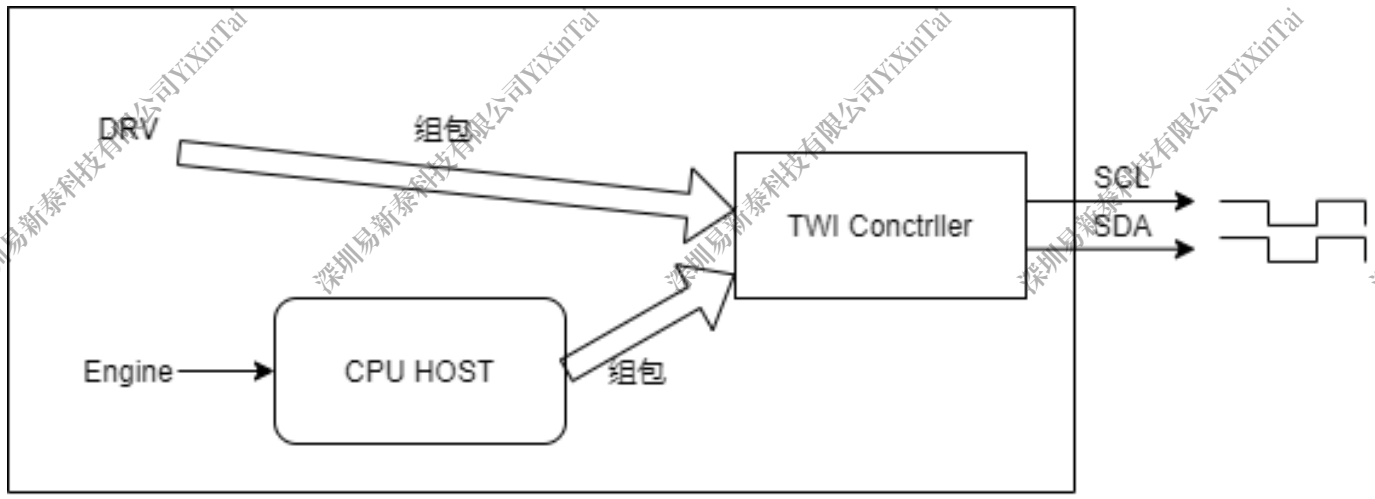


TWI 控制器的框图如上所示，该控制器支持的标准通信速率为 100Kbps，最高通信速率可以达到 400Kbps。其中 CPUX 域的 TWI 控制器时钟源来自于 APB2，CPUS 域的 S-TWI 时钟源来自于 APBS。

TWI 传输数据的方式包括 CPU 传输和 DMA 运输。

TWI 传输数据有两种模式：drv 模式和 engine 模式。其中 drv 模式是不需要经过 cpu host 组包和 conctrller 通信的。而 engine 模式则需要经过 cpu host 组包和 conctrller 通信。**drv 模式只支持做主机。**

图 2-2: drv 和 eng 区别



2.2 模块配置

2.2.1 sys_config.fex 配置说明

twi 的 io 配置如下。比如想配置 twi0，则配置主键为【twi0】，接着配置 sck 和 sda 的 io。

路径：rtos/board/{IC}/{BOARD}/configs/

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[twi0]
twi_sck  = port:PA03<4><1><default><default>
twi_sda  = port:PA04<4><1><default><default>
```

2.2.2 menuconfig 配置说明

```
Drivers Options  --->
    soc related device drivers  --->
        TWI Devices  --->
            [*] enable twi driver    #使能TWI驱动
            [ ]   enable twi hal APIs test command   #TWI驱动测试代码
```

2.3 源码结构

TWI 模块源码结构如下所示：

```
rtos-hal/
|--hal/source/twi/hal_twi.c       # hal层接口代码
|--hal/source/twi/common_twi.h
|--hal/source/twi/platform/       # 不同版型的配置文件
|--hal/source/twi/platform_twi.h
|--include/hal/sunxi_hal_twi.h    # 头文件
```

- hal_twi.c

这是 twi 的驱动源码

- hal/source/twi/platform/

这里是不同版型的 TWI 模块寄存器的基本配置。包含寄存器和中断号等。已 MR527 为例。

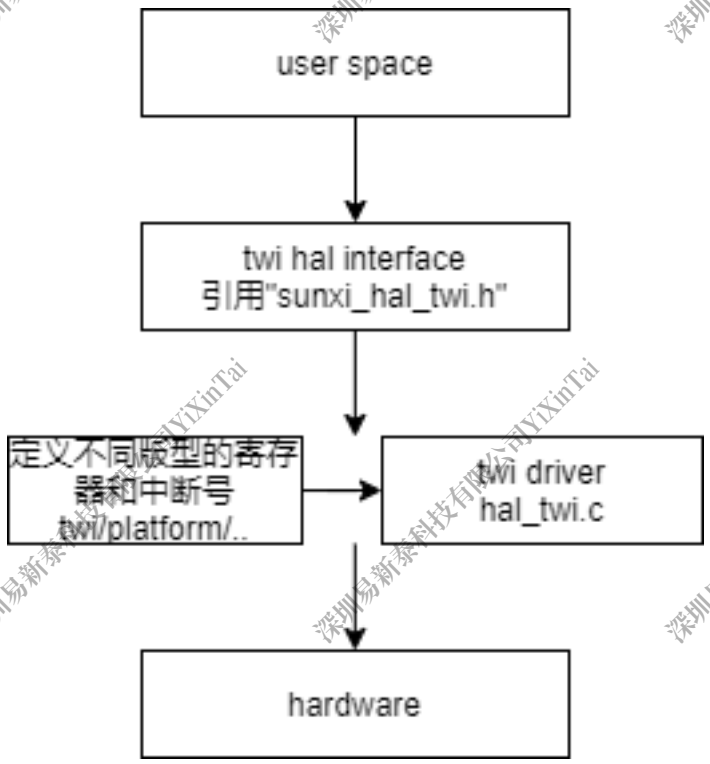
```
#if defined(CONFIG_ARCH_RISCV)  /* twi irq of RV */
#define SUNXI_IRQ_TWI0      MAKE_IRQn(91, 3)
#define SUNXI_IRQ_TWI1      MAKE_IRQn(91, 4)
#define SUNXI_IRQ_TWI2      MAKE_IRQn(91, 5)
#define SUNXI_IRQ_TWI3      MAKE_IRQn(91, 6)
#define SUNXI_IRQ_TWI4      MAKE_IRQn(91, 7)
#define SUNXI_IRQ_TWI5      MAKE_IRQn(91, 8)
#define SUNXI_IRQ_S_TWI0    MAKE_IRQn(68, 0)
#define SUNXI_IRQ_S_TWI1    MAKE_IRQn(69, 0)
#define SUNXI_IRQ_S_TWI2    MAKE_IRQn(60, 0)
// ....

/** the base address of TWI*/
#define SUNXI_TWI0_PBASE    0x02502000
#define SUNXI_TWI1_PBASE    0x02502400
#define SUNXI_TWI2_PBASE    0x02502800
#define SUNXI_TWI3_PBASE    0x02502c00
#define SUNXI_TWI4_PBASE    0x02503000
#define SUNXI_TWI5_PBASE    0x02503400
#define SUNXI_S_TWI0_PBASE  0x07081400
#define SUNXI_S_TWI1_PBASE  0x07081800
#define SUNXI_S_TWI2_PBASE  0x07081c00
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

2.4 软件框架

图 2-3: RTOS RTC 框架



主要软件框架如上图所示。用户层调用 sunxi_hal_twi.h 可以使用 twi 接口进行开发。hal_twi.c 文件实现了 twi 的驱动。向下对硬件层进行适配，向上提供 api。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 3 章 模块接口说明

表 3-1: 模块接口列表

API	解释说明
hal_twi_init	twi 模块初始化（freq 默认 400K）
hal_twi_init_with_freq	twi 模块初始化（freq 可配置）
hal_twi_uninit	twi 反初始化
hal_twi_reg_slave	配置从机模式，注册从机回调函数
hal_twi_unreg_slave	反注册从机回调函数
hal_twi_slave_reset	reset twi 从机
hal_twi_write	twi 写接口，需要配置好 twi_msg，在传进来
hal_twi_write_data	twi 写数据接口，不需要配置 twi_msg
hal_twi_read	twi 读接口
hal_twi_control	twi 控制接口
hal_twi_xfer	twi 传输接口（读写接口底层调用这个实现的）
hal_twi_get_mode	获取工作模式（drv/eng）
hal_twi_set_mode	设置工作模式（drv/eng）
hal_twi_get_freq	获取 scl 时钟（100k、200k、400k）
hal_twi_set_freq	设置 scl 时钟（100k、200k、400k）

3.1 数据结构

TWI 模块提供给用户配置的主要包括 TWI 的端口号，TWI 模块的一些控制用法以及发送的消息，所以重点讲解这几个数据结构，想要了解更多的数据结构可以查看 sunxi_hal_twi.h 文件。

3.1.1 TWI 端口号 twi_port_t

该数据结构主要用来表明 TWI 的编号，用户可以用来调用 TWI 的控制器。具体定义如下：

```
typedef enum
{
    TWI_MASTER_0,           /**< TWI master 0. */
    TWI_MASTER_1,           /**< TWI master 1. */
    TWI_MASTER_2,           /**< TWI master 0. */
    TWI_MASTER_3,           /**< TWI master 1. */
    TWI_MASTER_4,           /**< TWI master 4. */
    TWI_MASTER_5,           /**< TWI master 5. */
    S_TWI_MASTER_0,         /**< S_TWI master 0. */
    S_TWI_MASTER_1,         /**< S_TWI master 1. */
    S_TWI_MASTER_2,         /**< S_TWI master 2. */
    TWI_MASTER_MAX          /**< max TWI master number, \<invalid\> */
} twi_port_t;
```

3.1.2 TWI 消息结构体 twi_msg_t

该数据结构是 TWI 通信时的消息结构，定义每个通信数据的格式：

```
typedef struct twi_msg
{
    uint16_t addr;    /* 从设备地址 */
    uint16_t flags;    /* flag，可以定义的看下面两个宏 */
    #define TWI_M_RD 0x0001    /* read data, from slave to master（读） */
                               * TWI_M_RD is guaranteed to be 0x0001!（写）
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
                                */
    #define TWI_M_TEN 0x0010    /* this is a ten bit chip address （10位地址）*/
    uint16_t len;    /* msg length */
    uint8_t *buf;    /* pointer to msg data */
} twi_msg_t;
```

3.1.3 TWI 控制结构体 hal_twi_transfer_cmd_t

该数据接口储存了一些用户在调用 twi_control 的时候可以用到的一些参数，具体如下：

```
typedef enum
{
    TWI_SLAVE = 0,    /* 设置从机地址 */
    TWI_SLAVE_FORCE = 1,    /* 强制设置从机地址 */
    TWI_TENBIT = 2,    /* 支持10位地址 */
    TWI_RDWR = 3    /* 读写支持 */
} hal_twi_transfer_cmd_t;
```

3.2 TWI 初始化接口

- 原型：twi_status_t hal_twi_init(twi_port_t port)
- 功能：TWI 模块初始化，主要初始化时钟，中断以及引脚配置等
- 参数：
  - port：TWI 端口号
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.3 TWI 初始化接口（带 freq）

- 原型：twi_status_t hal_twi_init_with_freq(twi_port_t port, twi_frequency_t freq)
- 功能：TWI 模块初始化，主要初始化时钟，中断以及引脚配置等, 可以配置 freq
- 参数：
  - port：TWI 端口号
  - freq：配置 scl 频率
- 返回值：
  - 0 代表成功
  - 负数代表失败

 说明

hal_twi_init 和 hal_twi_init_with_freq 主要区别在于：前者默认时钟是 400k，后者可以配置时钟

3.4 TWI 反初始化接口

- 原型：twi_status_t hal_twi_uninit(twi_port_t port);
- 功能：TWI 模块反初始化，主要关闭中断，时钟，还原引脚等
- 参数：
  - port：TWI 端口号
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.5 TWI 配置从机模式接口

- 原型：twi_status_t hal_twi_reg_slave(twi_port_t port, uint16_t addr, uint8_t ten_bit_en, twi_frequency_t freq, i2c_slave_cb_t slave_cb);

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

- 功能：配置 TWI 成从机模式，调用这个接口，接着调用 TWI 初始化接口即可。可以注册回调函数。
- 参数：
  - port：TWI 端口号
  - addr：作为从机时的从机地址
  - ten_bit_en：1：10bit 模式地址；0：7bit 模式地址
  - freq：scl 时钟
  - slave_cb：回调函数
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.6 TWI 配置从机模式回调函数

- 原型：typedef int (i2c_slave_cb_t)(uint8_t port, enum i2c_slave_event event, u8 val);
- 功能：从机模式下和主机通信时读写数据，ack 等流程会触发回调函数。
- 参数：
  - port：TWI 端口号
  - event：twi 模块上报的事件，读写数据，ack 等。
  - val：交互时的数据
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.7 TWI 反配置从机模式接口

- 原型：twi_status_t hal_twi_unreg_slave(twi_port_t port);
- 功能：TWI 配置从机模式接口反函数。
- 参数：
  - port：TWI 端口号
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.8 TWI 从机模式重置接口

- 原型：twi_status_t hal_twi_unreg_slave(twi_port_t port);
- 功能：TWI 从机模式的 reset 接口。
- 参数：
  - port：TWI 端口号
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.9 TWI 数据发送接口 v1

- 原型：twi_status_t hal_twi_write(twi_port_t port, twi_msg_t *msg, uint32_t msg_num)
- 功能：用于发送数据
- 参数：
  - port：通道号
  - msg：封装好的 twi_msg_t 消息结构体指针
  - msg_num：twi_msg_t 消息结构体数量
- 返回值：
  - 0 代表成功

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

- 负数代表失败

3.10 TWI 数据发送接口 v2

- 原型：twi_status_t hal_twi_write_data(twi_port_t port, uint8_t read_addr, const void *buf, uint32_t size)
- 功能：用于发送数据
- 参数：
  - port：通道号
  - read_addr: 从机寄存器地址
  - buf：要传输的数据 buf
  - size：传输的数据长度
- 返回值：
  - 0 代表成功
  - 负数代表失败



说明

TWI 数据发送接口 v1 和 v2 的区别：
hal_twi_write 接口需要封装 twi_msg_t 消息结构体；
hal_twi_write_data 不需要封装 twi_msg_t 消息结构体，直接传入寄存器地址和 buf 就可以。

3.11 TWI 数据接收接口

- 原型：twi_status_t hal_twi_read(twi_port_t port, hal_twi_transfer_cmd_t cmd, uint16_t slave_addr, uint8_t reg_addr, void *buf, uint32_t size)
- 功能：用于接收数据
- 参数：
  - port：通道号
  - cmd：控制参数
  - slave_addr：从设备地址
  - reg_addr：寄存器地址
  - buf：接收的数据
  - size：接收数据大小，不包括偏移量

3.12 TWI 数据控制接口

- 原型：twi_status_t hal_twi_control(twi_port_t port, hal_twi_transfer_cmd_t cmd, void *args)
- 功能：可控制读写，配置地址模式，地址等
- 参数：
  - port：通道号
  - cmd: 指令，参考 hal_twi_transfer_cmd_t 结构体
  - args：指令配置参数
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.13 twi 传输接口

- 原型：twi_status_t hal_twi_xfer(twi_port_t port, twi_msg_t *msgs, int32_t num);
- 功能：读写接口都是调用这个函数实现的，用户层可以用这个函数自己实现读写函数。
- 参数：
  - port：通道号
  - msg：封装好的 twi_msg_t 消息结构体指针

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

- msg_num：twi_msg_t 消息结构体数量

- 返回值：
  - 0 代表成功
  - 负数代表失败

说明

开发过程中可以直接调用读写接口，api 提供的 twi 传输接口是为了支持用户层可以 diy 读写接口。

3.14 TWI 获取工作模式接口

- 原型：uint32_t hal_twi_get_mode(twi_port_t port)
- 功能：获取 twi 工作模式（drv、enginer）
- 参数：
  - port：TWI 端口号
- 返回值：twi 工作模式（drv、enginer）

3.15 TWI 设置工作模式接口

- 原型：twi_status_t hal_twi_set_mode(twi_port_t port, uint32_t mode);
- 功能：设置 twi 工作模式（drv、enginer）
- 参数：
  - port：TWI 端口号
- mode：1：drv；0：engine
- 返回值：
  - 0 代表成功
  - 负数代表失败

3.16 TWI 获取 scl 时钟接口

- 原型：uint32_t hal_twi_get_freq(twi_port_t port)
- 功能：获取 twi 时钟
- 参数：
  - port：TWI 端口号
- 返回值：twi scl 时钟（100k、200k、400k）

3.17 TWI 设置 scl 时钟接口

- 原型：twi_status_t hal_twi_set_freq(twi_port_t port, twi_frequency_t freq);
- 功能：设置 twi 时钟
- 参数：
  - port：TWI 端口号
  - freq：设置 scl 时钟
- 返回值：
  - 0 代表成功
  - 负数代表失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 4 章 功能开发

TWI 驱动提供了以下常用功能：

- twi 做为主机读写功能
- twi 做为从机读写功能

4.1 TWI 做为主机读写流程

4.1.1 开发流程

步骤 1 初始化 twi

```
hal_twi_init(port);
```

步骤 2 读写 twi 数据

1. 读

```
/* 申请一段buf来装twi返回的数据 */
uint8_t *buffer;
buffer = (uint8_t *)malloc(reg_val * sizeof(uint8_t));
if (g_addr_len) //设置成10bit模式的地址
    twi_status = hal_twi_read(port, I2C_TENBIT, addr, reg_addr, buffer, reg_val);
else            //设置成7bit模式的地址
    twi_status = hal_twi_read(port, I2C_SLAVE, addr, reg_addr, buffer, reg_val);
```

2. 写

```
buf = (uint8_t *)malloc((w_len + 1) * sizeof(uint8_t));//申请一段内存来发送数据
buf[0] = reg_addr;  //这段内存第一个字节要放从机寄存器地址
memset(buf + 1, reg_val, w_len); //w_len是要发送的数据长度，所以申请的时候要w_len+1

  if (g_addr_len)  //10bit模式
    msg.flags |= TWI_M_TEN;
  else             //7bit模式
    msg.flags &= ~TWI_M_TEN;
  msg.addr = addr;  //从机地址
  msg.len = w_len + 1;  //msg的长度
  msg.buf = buf;        //要写的数据buf

  twi_status = hal_twi_write(port, &msg, 1); //写数据接口
```

步骤 3 反初始化

```
hal_twi_uninit(port);
```

4.1.2 编程示例

参考路径：hal/test/twi/test_twi.c

4.2 TWI 做为从机读写流程

4.2.1 开发流程

步骤 1 初始化 twi

```
hal_twi_init(port);//初始化twi模块
hal_twi_reg_slave(port, addr, ten_bit_en, freq, i2c_slave_eeprom_slave_cb);//配置成从机模式，注册回调函数
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

步骤 2 中断回调处理

做为从机的时候是被动被主机触发，主机发出一些读写指令还有 ack 响应，都会触发回调。在回调里做数据交互处理。接下来的例子以 eeprom 来模拟。

```
struct twi_slave_eeprom_data {
    uint8_t buffer[256];
}; //申请一段内存模拟rom

static int i2c_slave_eeprom_slave_cb(uint8_t port, enum i2c_slave_event event, u8 *val)
{
    struct twi_slave_eeprom_data *eeprom = &twi_slave_eeprom[port];

    switch (event) {
    case I2C_SLAVE_WRITE_RECEIVED:
        // 收到主机写过来的数据，将数据保存到buf中
        eeprom->buffer[eeprom->buffer_idx++ & eeprom->address_mask] = *val;
        break;

    case I2C_SLAVE_READ_PROCESSED:
        //主机已经读取数据了，buf的地址相对移动，为主机要读取下一byte做准备
        eeprom->buffer_idx++;
    case I2C_SLAVE_READ_REQUESTED:
        //主机正在请求读数据，将buf中的数据写道val中。
        *val = eeprom->buffer[eeprom->buffer_idx & eeprom->address_mask];
        break;

    case I2C_SLAVE_STOP:
    case I2C_SLAVE_WRITE_REQUESTED:
        // 停止传输，或者主机刚开始请求写，将写计数清零
        eeprom->idx_write_cnt = 0;
        break;

    default:
        break;
    }

    return 0;
}
```

步骤 3 反初始化

```
    hal_twi_unreg_slave(port);
    hal_twi_uninit(port);
```

4.2.2 编程示例

参考路径：hal/test/twi/test_twi_slave_eeprom.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 5 章 日志分析

5.1 调试信息

twi 调试接口如下

路径：lichee/rtos/drivers/rtos-hal/include/hal/sunxi_hal_twi.h

```
#define CONFIG_DRIVERS_RTC_DEBUG
#ifdef CONFIG_DRIVERS_RTC_DEBUG
#define RTC_INFO(fmt, arg...) hal_log_info(fmt, ##arg)
#else
#define RTC_INFO(fmt, arg...) do {}while(0)
#endif
```

5.2 调试工具

- 配置 CONFIG_HAL_TEST_TWI 后，可以使用下面命令验证 twi 模块是否能正常工作。

1，配置 twi 模式（可配置频率，配置 10/7bit 地址模式）：

命令解释	说明
hal_twi_para -m 0	0：engine 模式；1：drv 模式
hal_twi_para -f [val]	配置频率：val: 1:100khz; 2:200khz; 4:400khz
hal_twi_para -b [val]	配置地址格式：val: 1:10BIT；0:7bit

2，twi 读写测试命令：

验证命令	解释说明
hal_twi [p1] [p2] [p3] -r [p4]	读指令：p1: 端口号；P2：从设备地址；P3：寄存器地址；P4：读数据长度
hal_twi [p1] [p2] [p3] -w [p4] [p5]	写指令：p1: 端口号；P2：从设备地址；P3：寄存器地址；p4: 写的值；p5：发送数据长度

3，twi 从机模拟 eeprom 命令

验证命令	解释说明
hal_twi_slave_driver probe 1 100000 0x50 1	初始化 twi 从机模式，选择 twi1 端口，设置 100khz，设置地址 0x50，使能 10bit 模式
hal_twi_slave_driver dump 1 0x00 5	把 twi1 端口收到的数据，从 0x00 地址，dump 出 5 个数据
hal_twi_slave_driver remove 1	反初始化 twi1 端口

```
uart>hal_twi_para  b 0
twi slave addr len: 7 Bit
hal_twi 0 0x50 0x00  w 6
uart>hal_twi_slave_driver probe 1 100000 0x50 1
# 这里开始测试10bit的读写
uart>hal_twi_para  b 1
twi slave addr len: 10 Bit
# 写入5byte的0x33
hal_twi 0 0x50 0x00  w 0x33 5
loop0: write: 0x33
Twi test finish
Twi test1 finish
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
# 读到5byte的0x33
uart>hal_twi 0 0x50 0x00  r 5
Run twi test
twi read test
loop0:
33 33 33 33 33
Twi test finish
Twi test1 finish
# 从机也能读到5byte的0x33
hal_twi_slave_driver dump 1 0x00 5
00000000: 33 33 33 33 33
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 6 章 FAQ

6.1 常见问题排查

在 twi 驱动异常的时候，会有一些 log，以及返回一些状态码。twi 内部有个寄存器返回状态码，这个寄存器可以查看一号通上的芯片手册上的 twi 章节的 “TWI Status Register” 寄存器。

图 6-1: 状态码寄存器

6.5. TWI Status Register(Default Value:0x0000_0000)

Offset: 0x0010			Register Name: TWI_STAT
Bit	Read/Write	Default/Hex	Description
31:8	/	/	/
7:0	R	0x0	STA Status Information Byte **decode status value** 0x00: Bus error 0x08: START condition transmitted 0x10: Repeated START condition transmitted 0x18: Address + Write bit transmitted, ACK received 0x20: Address + Write bit transmitted, ACK not received 0x28: Data byte transmitted in master mode, ACK received 0x30: Data byte transmitted in master mode, ACK not received 0x38: Arbitration lost in address or data byte 0x40: Address + Read bit transmitted, ACK received 0x48: Address + Read bit transmitted, ACK not received 0x50: Data byte received in master mode, ACK transmitted

接下来我们根据状态码和关键 log 来排查这些问题。

6.1.1 无法发送 start 信号

6.1.1.1 问题表现

log 中有关键字：

```
START can't sendout
```

6.1.1.2 可能原因和排查方法

1，原因 1：总线上有设备未处于正常工作状态，拉低 sck 和 sda

排查方法：用示波器或万用表测量 SCL 和 SDA 是否一直被拉低。排查一下硬件电路，有没有上拉电阻，上拉电源稳不稳定，更换外设排查外设是否有问题。

2，原因 2：总线上有高电平，可能配置有问题

排查方法：dump 出控制寄存器（0xc）的 BIT[6]，排查 BUS_EN 是否使能了。排查引脚配置，是否有其他引脚复用了 twi 引脚。排查 boot0，uboot 阶段是否使用了 twi 的这组引脚没有释放，可以在 kernel 起来时读一下 twi 这组引脚的 mapping。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

6.1.2 无 ack 应答

6.1.2.1 问题表现

log 中有关键字：

```
ACK not received
```

异常状态码

```
0x20/0x30/0x48/0x58/0xd8
```

6.1.2.2 可能原因和排查方法

1，原因 1：总线上无与此 slave address 匹配的设备

排查方法：检查开发板上是否有从设备连接

2, 原因 2：总线上有匹配的设备但该设备损坏或排线连接异常而无 ACK 响应

排查方法：可以反复重新插拔该从设备进行测试，更换排线和从设备进行测试

3, 原因 3：总线上与之匹配的设备在错误的时钟沿（时钟毛刺造成）已发送 ACK，而 master 在正确的时钟沿未收到 ACK.

排查方法：可以隔一段时间进行测试，看该问题是否是必现的；使用逻辑分析仪或者示波器抓取通信时的波形进行具体分析

6.1.3 无法发送 stop 信号

6.1.3.1 问题表现

log 中有关键字：

```
STOP can't sendout!
```

6.1.3.2 可能原因和排查方法

1，原因 1：上拉电阻未贴片

排查方法：检查开发板对应位置是否有上拉电阻

6.1.4 总线处于非空闲状态

6.1.4.1 问题表现

log 中有关键字：

```
i2c state isn’t idle
```

6.1.4.2 可能原因和排查方法

1，原因 1：twi 引脚配置出错：io 耐压值、引脚驱动能力、引脚上下拉等配置出错

排查方法：用万用表量 SDA 与 SCL 电压，检查 SDA 和 SCL 的电压是否正常，波动情况是否正常。检查 sysconfig 文件中引脚名、驱动能力、上下拉等是否正确配置

6.2 一号通 FAQ 列表

- FAQ1887

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- FAQ2817
- FAQ2640

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2026 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。